

Joga Aí

Sistema de Empréstimo de Jogos de Tabuleiro

Sprint 3 – Documentação Técnica de Implementação

Disciplina:	DCC603 – Engenharia de Software I
Instituição:	Universidade Federal de Minas Gerais (UFMG)
Professor:	Eduardo Figueiredo
Tema:	11 – Sistema de Controle de Empréstimo de Jogos
Equipe:	Álvaro Lima, Beatriz Nogueira, Lucas Dolabella, Sarah Menks, Vanessa Nascimento

1. Visão Geral do Sistema

O Joga Aí é uma plataforma web que conecta pessoas interessadas em emprestar e pegar emprestados jogos físicos, como jogos de tabuleiro e cartas. O sistema foi desenvolvido como trabalho prático da disciplina DCC603 – Engenharia de Software I, seguindo o processo ágil SCRUM com desenvolvimento incremental ao longo de 3 sprints.

O objetivo central é permitir que usuários cadastrem seus jogos disponíveis, pesquisem jogos de outros usuários, negociem prazos via chat interno e gerenciem o ciclo completo de empréstimo, incluindo controle de prazos, notificações, penalidades por atraso e desbloqueio mediante pagamento de multa.

1.1 Tecnologias Utilizadas

Componente	Tecnologia	Justificativa
Backend	Python 3 + Flask	OOP nativo, estrutura modular clara, REST simples
ORM / Dados	SQLAlchemy + SQLite	Sem setup, arquivo único, ORM robusto
Autenticação	JWT (PyJWT)	Stateless, ideal para APIs REST
Senhas	Werkzeug (bcrypt)	Hash seguro incluído no Flask
Frontend	HTML5 + CSS3 + JS	SPA vanilla, sem build step, zero dependências

1.2 Como Executar

```
# 1. Criar e ativar ambiente virtual
python3 -m venv venv
source venv/bin/activate

# 2. Instalar dependências
pip install -r requirements.txt

# 3. Iniciar o servidor
python run.py

# 4. Abrir no navegador
http://localhost:5000
```

2. Arquitetura do Sistema

O sistema adota a Arquitetura em Camadas (Layered Architecture), definida na Sprint 2, organizada em cinco camadas com responsabilidades bem separadas.

2.1 Camadas e Correspondência com o Código

Camada	Diretório	Responsabilidade
Apresentação	templates/ + static/	Frontend SPA: HTML, CSS, JavaScript
Aplicação	app/routes/	API REST – recebe requisições, valida, repassa
Negócio	app/models/ + app/services/	Classes OOP + regras de negócio
Dados	app/database.py + SQLite	Persistência via SQLAlchemy ORM
Sistemas Externos	app/external/	E-mail e pagamento (stubs integráveis)

2.2 Estrutura de Diretórios

```
TP-Engenharia-de-Software-I/
├── run.py                # Ponto de entrada
├── requirements.txt      # Flask, SQLAlchemy, PyJWT, Werkzeug
└── app/
```

```

├── __init__.py          # App factory (create_app)
├── database.py          # Instância do SQLAlchemy
├── models/              # Camada de Negócio - Classes OOP
│   ├── usuario.py      # Classe Usuario + StatusUsuario
│   ├── jogo.py         # Classe Jogo + StatusJogo
│   ├── emprestimo.py   # Classe Emprestimo + StatusEmprestimo
│   ├── mensagem.py     # Classe Mensagem (chat)
│   ├── notificacao.py  # Classe Notificacao + TipoNotificacao
│   └── penalidade.py   # Classe Penalidade + StatusPenalidade
├── services/            # Camada de Aplicação - Serviços
│   ├── auth_service.py
│   ├── jogo_service.py
│   ├── emprestimo_service.py
│   ├── chat_service.py
│   └── penalidade_service.py
├── routes/              # Camada de Apresentação - API REST
│   ├── auth.py
│   ├── jogos.py
│   ├── emprestimos.py
│   ├── chat.py
│   ├── penalidades.py
│   └── notificacoes.py
└── external/            # Sistemas Externos
    ├── email_service.py
    └── payment_service.py
├── templates/index.html # SPA principal
├── static/css/style.css
└── static/js/           # api.js, auth.js, jogos.js, emprestimos.js,
                        # chat.js, app.js

```

3. Classes OOP – Implementação do Diagrama de Classes

As seis classes principais do sistema foram implementadas diretamente a partir do Diagrama de Classes definido na Sprint 2, mantendo os mesmos atributos, métodos, visibilidades e enumerações. Cada classe estende db.Model (SQLAlchemy), unindo persistência e comportamento em um único objeto orientado a objetos.

3.1 Usuario (app/models/usuario.py)

Atributo / Método	Tipo	Descrição
id	String (UUID)	Identificador único gerado automaticamente
nome	String	Nome completo do usuário
email	String (único)	E-mail usado no login
senha	String	Hash bcrypt da senha
status	StatusUsuario	ATIVO / INATIVO / BLOQUEADO
avaliacao	Float	Média de avaliações recebidas
data_cadastro	DateTime	Data de criação da conta
cadastrar()	void	Inicializa e salva o usuário com senha hasheada
login()	bool	Valida e-mail e senha
bloquear()	void	Muda status para BLOQUEADO
desbloquear()	void	Muda status para ATIVO
_validar_senha()	bool	Método privado: verifica hash da senha

3.2 Jogo (app/models/jogo.py)

Atributo / Método	Tipo	Descrição
id	String (UUID)	Identificador único
nome	String	Nome do jogo
descricao	Text	Descrição completa
categoria	String	Ex: Estratégia, Cartas, RPG
status	StatusJogo	DISPONIVEL / EMPRESTADO
dono_id	FK -> Usuario	Usuário dono do jogo
cadastrar()	void	Valida e inicializa o jogo
verificar_disponibilidade()	bool	Retorna true se status = DISPONIVEL
atualizar_status()	void	Muda o status do jogo
buscar(termo)	List<Jogo>	Busca por nome ou categoria (ILIKE)
_validar_dados()	bool	Método privado: valida campos obrigatórios

3.3 Empréstimo (app/models/emprestimo.py)

Atributo / Método	Tipo	Descrição
id	String (UUID)	Identificador único
dono_id	FK -> Usuario	Dono do jogo
solicitante_id	FK -> Usuario	Quem pegou emprestado
jogo_id	FK -> Jogo	Jogo emprestado
status	StatusEmprestimo	ATIVO / DEVOLVIDO / CANCELADO
data_inicio	DateTime	Data da solicitação
data_prazo	DateTime	Prazo acordado para devolução
data_devolvido	DateTime	Data real de devolução (nullable)

solicitar()	void	Valida e registra o empréstimo
confirmar_devolucao()	void	Marca devolvido e libera o jogo
cancelar()	void	Cancela e libera o jogo
verificar_atraso()	bool	True se data atual > data_prazo e ATIVO
calcular_dias_atraso()	int	Dias entre prazo e hoje
_validar_solicitacao()	bool	Privado: verifica bloqueio, disponibilidade

3.4 Mensagem (app/models/mensagem.py)

Atributo / Método	Tipo	Descrição
id	String (UUID)	Identificador único
remetente_id	FK -> Usuario	Quem enviou a mensagem
destinatario_id	FK -> Usuario	Quem recebe
emprestimo_id	FK -> Empréstimo	Chat vinculado ao empréstimo
conteudo	Text	Texto da mensagem
lida	Boolean	Se o destinatário já leu
data_envio	DateTime	Momento do envio
enviar()	void	Valida e prepara a mensagem
marcar_como_lida()	void	Marca lida = True
listar_por_emprestimo()	List	Retorna histórico ordenado por data
_validar_conteudo()	bool	Privado: impede mensagem vazia

3.5 Notificacao (app/models/notificacao.py)

Atributo / Método	Tipo	Descrição
id	String (UUID)	Identificador único
destinatario_id	FK -> Usuario	Quem recebe a notificação
mensagem	Text	Texto gerado automaticamente
tipo	TipoNotificacao	SOLICITACAO_EMPRESTIMO, PRAZO_VENCIDO, DEVOLUCAO_CONFIRMADA, NOVA_MENSAGEM, PENALIDADE_APL
lida	Boolean	Se o usuário já viu
data_envio	DateTime	Momento do envio
emprestimo_id	FK -> Empréstimo	Empréstimo relacionado (opcional)
enviar()	void	Inicializa e gera a mensagem pelo tipo
marcar_como_lida()	void	Marca lida = True
_gerar_mensagem()	str	Privado: texto descritivo por tipo

3.6 Penalidade (app/models/penalidade.py)

Atributo / Método	Tipo	Descrição
id	String (UUID)	Identificador único
usuario_id	FK -> Usuario	Quem recebeu a penalidade
emprestimo_id	FK -> Empréstimo	Empréstimo em atraso
valor_multa	Float	R\$ 5,00 por dia de atraso
status	StatusPenalidade	PENDENTE / PAGA / CANCELADA
data_aplicacao	DateTime	Quando a penalidade foi gerada
data_pagamento	DateTime	Quando foi quitada (nullable)
aplicar()	void	Calcula multa e bloqueia o usuário
confirmar_pagamento()	void	Marca PAGA e desbloqueia o usuário
desbloquear_usuario()	void	Chama usuario.desbloquear()
_calcular_multa()	float	Privado: dias_atraso × R\$5,00 (mínimo R\$5)
_bloquear_usuario()	void	Privado: chama usuario.bloquear()

4. API REST – Endpoints

4.1 Autenticação (/api/auth)

Método	Endpoint	Auth?	Descrição
POST	/api/auth/cadastrar	Não	Registra novo usuário, retorna JWT
POST	/api/auth/login	Não	Login com e-mail e senha, retorna JWT
GET	/api/auth/perfil	Sim	Retorna dados do usuário autenticado
PUT	/api/auth/perfil	Sim	Atualiza nome ou senha

4.2 Jogos (/api/jogos)

Método	Endpoint	Auth?	Descrição
GET	/api/jogos?busca=termo	Não	Lista todos os jogos (com busca opcional)
GET	/api/jogos/meus	Sim	Lista jogos do usuário autenticado
GET	/api/jogos/:id	Não	Detalhe de um jogo
POST	/api/jogos	Sim	Cadastra novo jogo
PUT	/api/jogos/:id	Sim	Atualiza jogo (somente dono)
DELETE	/api/jogos/:id	Sim	Remove jogo (somente dono, se disponível)

4.3 Empréstimos (/api/emprestimos)

Método	Endpoint	Auth?	Descrição
GET	/api/emprestimos	Sim	Lista empréstimos do usuário (dono ou solicitante)
GET	/api/emprestimos/:id	Sim	Detalhe de um empréstimo
POST	/api/emprestimos	Sim	Solicita empréstimo de um jogo
PUT	/api/emprestimos/:id/confirmar-devolucao	Sim	Dono confirma devolução
PUT	/api/emprestimos/:id/nao-devolvido	Sim	Dono reporta não devolução
PUT	/api/emprestimos/:id/cancelar	Sim	Cancela empréstimo ativo

4.4 Chat, Penalidades e Notificações

Método	Endpoint	Auth?	Descrição
GET	/api/chat/:emprestimo_id	Sim	Lista mensagens do chat (marca como lidas)
POST	/api/chat/:emprestimo_id	Sim	Envia mensagem no chat
GET	/api/penalidades	Sim	Lista penalidades do usuário
POST	/api/penalidades/:id/pagar	Sim	Processa pagamento e desbloqueia
GET	/api/notificacoes	Sim	Lista notificações do usuário
PUT	/api/notificacoes/:id/lida	Sim	Marca notificação como lida
PUT	/api/notificacoes/marcar-todas-lidas	Sim	Marca todas como lidas

5. Fluxos Implementados

5.1 Fluxo de Empréstimo

Passo	Ator	Ação
1	Solicitante	Acessa 'Explorar', encontra jogo disponível
2	Solicitante	Clica em 'Pegar Emprestado', define prazo de devolução
3	Sistema	Valida: usuário não bloqueado, jogo disponível, não é o próprio dono
4	Sistema	Cria Empréstimo (ATIVO), muda status do Jogo para EMPRESTADO
5	Sistema	Cria Notificacao para o dono (SOLICITACAO_EMPRESTIMO)
6	Ambos	Podem trocar mensagens via Chat (atualização a cada 4s)
7	Sistema	No vencimento: envia e-mail ao dono (stub EmailService)
8	Dono	Confirma devolução → Empréstimo DEVOLVIDO, Jogo DISPONIVEL
8b	Dono	Ou reporta não devolução → Penalidade aplicada, solicitante BLOQUEADO

5.2 Fluxo de Penalidade e Pagamento

Passo	Ator	Ação
1	Sistema	Dono clica em 'Não Devolvido'
2	Sistema	Calcula dias de atraso × R\$5,00 (mínimo R\$5,00)
3	Sistema	Cria Penalidade (PENDENTE), bloqueia solicitante
4	Sistema	Cria Notificacao PENALIDADE_APLICADA, envia e-mail
5	Solicitante	Acessa 'Penalidades', vê valor e motivo
6	Solicitante	Clica em 'Pagar Multa'
7	Sistema	PaymentService.processar() → aprovado
8	Sistema	Penalidade.confirmar_pagamento() → status PAGA
9	Sistema	usuario.desbloquear() → status ATIVO
10	Sistema	Envia comprovante por e-mail (stub)

6. Requisitos Implementados

6.1 Requisitos Funcionais

RF	Descrição	Status
RF01	Cadastro de usuários (nome, e-mail, senha)	Implementado
RF02	Login / Logout com autenticação JWT	Implementado
RF03	Cadastrar jogos disponíveis para empréstimo	Implementado
RF04	Visualizar jogos disponíveis na plataforma	Implementado
RF05	Busca de jogos por nome ou categoria	Implementado
RF06	Solicitar empréstimo de um jogo disponível	Implementado
RF07	Definir prazo para devolução	Implementado
RF08	Registrar todas as informações do empréstimo	Implementado
RF09	Chat interno vinculado ao empréstimo	Implementado
RF10	E-mail ao dono no vencimento do prazo	Stub (EmailService)
RF11	Dono confirma devolução do jogo	Implementado
RF12	Bloquear usuário inadimplente	Implementado
RF13	Aplicar multa por atraso (R\$5/dia)	Implementado
RF14	Desbloquear após pagamento de multa	Implementado
RF17	Status do jogo (disponível / emprestado)	Implementado
RF18	Cancelar solicitação de empréstimo	Implementado
RF19	Notificações internas no sistema	Implementado

6.2 Requisitos Não Funcionais Atendidos

RNF	Descrição	Como foi atendido
RNF01	Facilidade de uso	Interface SPA intuitiva, sem treinamento
RNF05	Proteção de dados	JWT + bcrypt para senhas
RNF06	Criptografia de senha	Werkzeug generate_password_hash
RNF07	Autenticação restrita	Middleware requer_autenticacao em todas as rotas
RNF08	Integridade dos dados	Transações SQLAlchemy + FK constraints
RNF09	Consistência de estados	Um jogo não pode ser DISPONIVEL e EMPRESTADO
RNF12	Manutenibilidade	Arquitetura em camadas, módulos independentes
RNF13	Acesso multiplataforma	Aplicação web acessível em qualquer browser

7. Sistemas Externos (Stubs)

Os sistemas externos foram implementados como stubs (simulações) que registram as operações no log da aplicação. Em ambiente de produção, esses stubs seriam substituídos por integrações reais sem alterar nenhuma outra camada do sistema.

7.1 EmailService (app/external/email_service.py)

Método	Quando é chamado	Em produção
enviar_aviso_prazo_vencido()	Prazo do empréstimo vence	SMTP / SendGrid
enviar_confirmacao_devolucao()	Dono confirma devolução	SMTP / SendGrid
enviar_aviso_penalidade()	Penalidade aplicada	SMTP / SendGrid
enviar_comprovante_pagamento()	Multa paga	SMTP / SendGrid

7.2 PaymentService (app/external/payment service.py)

Método	Comportamento atual	Em produção
processar(valor, metodo)	Sempre retorna 'aprovado'	Stripe / PagSeguro / MercadoPago

8. Banco de Dados

O banco de dados utilizado é o SQLite, armazenado automaticamente no arquivo instance/jogaai.db ao iniciar a aplicação. O schema é criado pelo SQLAlchemy com db.create_all() na inicialização.

8.1 Tabelas

Tabela	Colunas principais	Relacionamentos
usuarios	id, nome, email, senha, status, avaliacao	1:N com jogos, emprestimos, penalidades
jogos	id, nome, descricao, categoria, status, dono_id	N:1 usuarios, 1:N emprestimos
emprestimos	id, dono_id, solicitante_id, jogo_id, status, datas	N:1 usuarios/jogos, 1:N mensagens
mensagens	id, remetente_id, destinatario_id, emprestimo_id, conteu	N:1 usuarios/emprestimos
notificacoes	id, destinatario_id, mensagem, tipo, lida, emprestimo_id	N:1 usuarios/emprestimos
penalidades	id, usuario_id, emprestimo_id, valor_multa, status	N:1 usuarios/emprestimos

8.2 Como visualizar os dados

Instale o DB Browser for SQLite e abra o arquivo instance/jogaai.db:

```
# Instalar via Homebrew (macOS)
brew install db-browser-for-sqlite

# Abrir
open -a 'DB Browser for SQLite'
# Arquivo: TP-Engenharia-de-Software-I/instance/jogaai.db

# Ou via terminal
sqlite3 instance/jogaai.db
.tables
SELECT * FROM usuarios;
SELECT * FROM emprestimos;
```

9. Frontend – SPA (Single Page Application)

O frontend é uma SPA (Single Page Application) construída com HTML, CSS e JavaScript puro (vanilla), sem frameworks externos. Todas as telas são renderizadas dinamicamente por JavaScript no mesmo arquivo index.html.

9.1 Telas Implementadas

Tela	Arquivo JS	Funcionalidades
Login / Cadastro	auth.js	Formulários de login e registro com JWT
Explorar Jogos	jogos.js (Jogos)	Grid de jogos, busca em tempo real, solicitar empréstimo
Meus Jogos	jogos.js (MeusJogos)	CRUD de jogos do usuário autenticado
Empréstimos	emprestimos.js	Lista empréstimos, confirmar/cancelar/reportar
Chat	chat.js	Mensagens em tempo real (polling 4s)
Penalidades	emprestimos.js	Listar e pagar multas pendentes
Notificações	emprestimos.js	Centro de notificações com badge no navbar

9.2 Comunicação com o Backend

Todas as chamadas ao backend são feitas via Fetch API (JavaScript nativo) através do objeto Api definido em static/js/api.js. O token JWT é armazenado no localStorage e enviado automaticamente no header Authorization: Bearer <token> em todas as requisições autenticadas.